

const in C++ — Complete Guide

1. const with Variables

A `const` variable cannot be modified after initialization. Must be initialized at declaration.

```
const int x = 10;
x = 20;           // ERROR: cannot assign to const
const int y;      // ERROR: must be initialized

const float PI = 3.14159f;
const char GRADE = 'A';
const bool FLAG = true;
```

2. const with Pointers — 3 Combinations

Form 1 — Pointer to const: `const int* ptr`

Data being pointed to cannot change. Pointer itself can move.

```
int a = 10, b = 20;
const int* ptr = &a;
*ptr = 50;    // ERROR
ptr = &b;     // OK

// Same as: int const* ptr
```

Form 2 — Const pointer: `int* const ptr`

Pointer cannot move. Data CAN change.

```
int a = 10, b = 20;
int* const ptr = &a;
*ptr = 50;    // OK
ptr = &b;     // ERROR
int* const p; // ERROR - must initialize
```

Form 3 — Const pointer to const: `const int* const ptr`

Nothing can change.

```
const int* const ptr = &a;
*ptr = 50;    // ERROR
ptr = &b;     // ERROR
```

Reading Trick — "Right to Left"

```
const int *      ptr    -> ptr is a pointer to int const      (data const)
int *      const ptr    -> ptr is a const pointer to int      (pointer const)
const int * const ptr    -> ptr is a const pointer to int const (both const)
```

Summary Table

Declaration	*ptr (data)	ptr (address)
int* ptr	mutable	mutable
const int* ptr	const	mutable
int* const ptr	mutable	const
const int* const ptr	const	const

Real-World Uses for `const int* const`

```
// 1. Hardware register at fixed address
const int* const HW_STATUS = (const int*)0x40021000;

// 2. Fixed lookup table
const char* const DAYS[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

// 3. Secret/config constants
static const char* const DB_PASSWORD = "s3cr3t!";
```

3. `const` with References

A reference is already a permanent alias. `const` on a reference means **the data cannot be modified through it**.

```
int x = 5;
const int& cref = x;
cref = 99;    // ERROR
x = 99;       // OK - x itself isn't const, cref sees the change
```

Special Power — `const&` binds to temporaries

```
const int& r1 = 42;    // OK - compiler extends temp's lifetime
int& r2 = 42;         // ERROR - non-const can't bind to temp
```

Behind the scenes:

```
// const int& r1 = 42 becomes:
const int __temp = 42;    // hidden stack temp
const int& r1 = __temp;   // r1 binds to it
// __temp's lifetime is extended to match r1's
```

Most Common Use — Function Parameters

```
void print(const string& s); // no copy, no modification, accepts literals
print("Hello");             // OK - literal works because of const
```

4. `const` Member Functions

Promises not to modify any member variables.

```
class Car {
    float speed;
    float fuel;
public:
    void Dashboard() const {           // const function
        cout << speed;               // OK - reading
        speed = 0;                   // ERROR - cannot modify
    }
    void Accelerate() {               // non-const function
        speed++;
    }
};

const Car c;
c.Dashboard();    // OK - const function on const object
c.Accelerate();   // ERROR - non-const function on const object
```

What the Compiler Actually Does

```
// What you write:
void Dashboard() const { cout << speed; }
```

```
// What compiler generates:
void Dashboard(const Car* this) { cout << this->speed; }
//          ^^^^^^^^^^^^^^^^^
// const Car* means this is const - cannot modify members
```

The mutable Exception

```
class Logger {
    mutable int callCount = 0;    // can change even in const function
public:
    string getData() const {
        callCount++;              // OK - mutable bypasses const
        return data;
    }
};
```

5. Dangling References — Why `int& r2 = 42` Fails

```
int& r2 = 42;    // ERROR - compiler blocks this
```

Why?

- `42` is a temporary (rvalue) - lives momentarily, no stable address
- Non-const reference would bind to the temp
- Temp dies immediately at `;` (no lifetime extension for non-const refs)
- `r2` would be a **dangling reference** pointing to freed memory

Why `const int& r1 = 42` works?

- C++ has a special rule: const references binding to temporaries extend the temp's lifetime to match the reference
- Allowed only for `const` because non-const would let you modify the temp (pointless and dangerous)

6. `constexpr` — Compile-Time Constant

```
const int x = input;          // OK - evaluated at runtime
constexpr int MAX = 100;     // MUST be compile-time constant

constexpr int square(int n) { return n * n; }
constexpr int result = square(5); // computed at compile time = 25

// constexpr is required for array sizes (technically)
```

```
constexpr int SIZE = 10;
int arr[SIZE];
```

7. constexpr and constinit (C++20)

```
// constexpr - MUST run at compile time
constexpr int getMax() { return 100; }
constexpr int a = getMax(); // OK
int b = getMax();           // ERROR - must be compile time

// constinit - compile-time init, but still mutable
constinit int counter = 0;
counter = 5;                // OK - not const
```

8. const Class Members

```
class Circle {
    const float PI = 3.14159f;
    const int id;           // must init in initializer list
    float radius;
public:
    Circle(int i, float r) : id(i), radius(r) {} // OK
    void setId(int i) { id = i; }                // ERROR
};
```

9. const Overloading — Two Versions

```
class MyArray {
    int data[10];
public:
    int& operator[](int i) { return data[i]; } // non-const
    const int& operator[](int i) const { return data[i]; } // const
};

MyArray arr;
arr[0] = 5; // calls non-const - can modify

const MyArray arr2;
cout << arr2[0]; // calls const - read-only
arr2[0] = 5;     // ERROR - const version returns const ref
```

10. Top-Level vs Low-Level `const`

```
// TOP-LEVEL - the variable/pointer itself is const
int* const ptr = &x;
const int val = 5;

// LOW-LEVEL - what is pointed to/referenced is const
const int* ptr2 = &x;
const int& ref = x;
```

Copying drops top-level, keeps low-level:

```
const int a = 5;
int b = a;           // OK - top-level const dropped

const int* p = &a;
int* p2 = p;         // ERROR - low-level const can't be dropped
```

11. `const` with `auto`

```
const int x = 10;
auto a = x;           // int (top-level const DROPPED by auto)
const auto b = x;     // const int (must add const explicitly)

auto& c = x;          // const int& (low-level const KEPT)
const auto& d = x;    // const int&
```

12. `const` with Iterators

```
vector<int> v = {1, 2, 3};

vector<int>::iterator it = v.begin();           // can read+modify
vector<int>::const_iterator cit = v.cbegin();   // read-only
*cit = 99;   // ERROR

// Modern way
for (const auto& val : v) { ... } // read-only - best for reading
for (auto& val : v) { val = 0; }  // modify
```

13. `const_cast` — Removing `const` (DANGEROUS)

```
// Only safe to remove const from something NOT originally declared const
void legacyFunc(int* p); // old API, not const-correct

const int x = 5;
legacyFunc(const_cast<int*>(&x)); // risky - undefined behavior if legacyFunc
modifies

// NEVER do this:
const int y = 10;
int* p = const_cast<int*>(&y);
*p = 20; // UNDEFINED BEHAVIOR - y was truly const
```

14. Complete `const` Map

SYNTAX	MEANING
<code>const int x = 5</code>	value never changes
<code>const int* p</code>	*p can't change
<code>int* const p</code>	p can't change
<code>const int* const p</code>	nothing can change
<code>const int& r</code>	read-only alias, binds to temps
<code>const int id</code>	set once in constructor
<code>void f() const</code>	member fn won't modify object
<code>mutable int x</code>	can change in const member fn
<code>constexpr int N = 10</code>	compile-time constant
<code>constexpr int f()</code>	must be compile-time
<code>constexpr int x = 0</code>	compile-time init only

Golden Rules

1. Mark everything `const` that CAN be `const`
2. Use `const&` for function parameters when reading large objects
3. Use `const` member functions for getters / read-only ops
4. `const` + reference + temporary = lifetime extension (special rule)
5. Never `const_cast` away true `const` - undefined behavior
6. Top-level `const` drops on copy, low-level `const` stays